

AI Usage Disclosure

Phase 3 — The AI Bidding War

Asli Gulcur · CMU Agentic Systems Studio · Track A · April 2026

This file describes **generative AI and assistant tools** used in **Phase 3** of *The AI Bidding War*, per course expectations: what was used, for what purpose, what was edited manually, and what was **independently verified**. Phase 1 and Phase 2 disclosures (Phase_1_Gen_AI_Use_-_Asli_Gulcur.pdf, Phase_2_AI_Usage.pdf) cover AI usage during ideation, architecture, and initial build; this document focuses on **Phase 3-specific** work: evaluation, failure-mode analysis, log-driven design revisions, and final report synthesis.

Human-led scope (Phase 3 continuity)

Phase 2 established the governance stance: I drove design, architecture, and governance decisions (hub-and-spoke isolation, Strategic Audit independence, deterministic Python guardrails, human-only nudge authority, semi-autonomous operation over full end-to-end autonomy). That stance did **not** change in Phase 3.

What **did** change in Phase 3 is where my time went. **I was hands-on with the evaluation process**, and that is where the bulk of Phase 3 human effort sits: running scenarios, inspecting logs, diagnosing emergent behaviors, identifying leakage and design gaps, and revising the system in response. I was also deeply hands-on with the evaluation report generation, because the initial AI-drafted outputs did not meet the analytical or presentational bar I wanted.

Phase 3 work I drove directly (not AI-generated)

The items below are findings I reached by reading logs and reasoning about cause and effect. **Cursor could not produce this analysis** — the causal links, the distinction between model variance and guardrail regressions, and the design implications are my own. AI tools assisted with articulation and drafting after the analysis was done; they did not do the analysis itself.

1. Scenario-by-scenario evaluation and leakage discovery

I ran the scenarios in `config/scenarios.json` independently, read the evidence logs, and inspected transcripts turn by turn. From that work I identified:

- **A structural cross-vendor leakage issue** in the buyer assistant path: the hub routes identical buyer text to vendors A/B/C as `buyer_reply_prev`, while the buyer's reasoning is fed by all vendors' parsed bids via `bid_summary`. Per-thread isolation does **not** stop commercial leakage if the buyer's reply quotes, compares, or ranks offers in prose — all vendors then see that intelligence.

- **Award-adjacent copy slippage:** the model could stay vendor-neutral mid-run, then put outcome pricing in prose in the same message as `AWARD_JSON`, which is still broadcast.
- **Scenario-level contamination of the public RFP:** earlier scenario drafts mixed internal steering, design notes, and "do not disclose" lines into requirements, which is the **public** RFP passed to all vendors. This occasionally produced vendor meta-commentary on leaked instructions.

To mitigate these findings, I consulted and brainstormed with Cursor; Cursor implemented the mitigations under my direction. The resulting changes — the public-broadcast rules in `agents/buyer/agent.md`, the `_PUBLIC_BROADCAST_BUYER_APPEND` duplication in `orchestrator.py` (so rules are not dropped when editing YAML alone), the `scenario_relax_public_broadcast` flag, and the separation of requirements (public RFP) from `buyer_private_brief` (buyer hub only) — are documented in full in `docs/Future Roadmap/PHASE_3_IMPLEMENTATION.md`. Scenario 11 (COI) is a representative example: pilot/dinner/relationship facts sit in `buyer_private_brief`, while requirements references familiarity only at a generic level.

2. Diagnosing the Sonnet snapshot effect on red-team scenarios

When the repository default model changed from `claude-sonnet-4-20250514` to `claude-sonnet-4-6` (updated across `.env`, `orchestrator.py`, `Dockerfile`, `docker-compose.yml`, and `agents/*/agent.md` meta), **I noticed that red-team scenarios stopped behaving the way they had previously.** Buyer and vendor negotiation phrasing shifted, emergent constructs (multi-year ladders, "best and final" scaffolding) appeared at different rates, red-team roleplay consistency changed (including more frequent transparency refusals), and general tone moved — all with the same `config/scenarios.json` and prompts.

I confirmed `award_contract` and scenario budgets were unchanged, so the observed differences were LLM variance. For traceability, each run's `model` field in `evidence_log_*.json` records which snapshot produced it; for apples-to-apples comparison across runs, `ANTHROPIC_MODEL` must be pinned.

3. Failure-mode analysis

The failure log in `evaluation/failure_log.md` (including FL-003 for the scenario 6 \$17k budget vs \$15k hard ceiling `NO_AWARD` pattern, and FL-004 closure) reflects the same pattern: **I built the causal chains.** Cursor could draft scaffolding text, but identifying what qualified as a failure vs. expected emergent behavior, distinguishing governance success (`BLOCKED / NO_AWARD` under an over-ceiling attempt) from model failure (refusal breaking the red-team arc), and deciding which patterns warranted a logged failure ID vs. a documentation note — those judgments are mine.

4. Emergent multi-year pricing — identifying it as emergent, not scripted

Runs sometimes showed "multi-year discount options," 2-year/3-year annual rates, or term ladders. I traced this through requirements, agents/buyer/agent.md, and the orchestrator's per-round buyer_user block and confirmed none of them instruct this behavior — it materializes as completion behavior on the buyer's assistant turn and vendors echo it in their bids. I also suspect this is tied to the Sonnet 4-6 snapshot change in finding 2, since I do not recall seeing multi-year constructs in Phase 2 runs, but I did not run a controlled comparison so this remains a hypothesis.

5. Governance nudge behavior characterization

I characterized the observed variance in how the buyer model responds to a human governance nudge (full alignment, partial compliance, resistance, or NO_AWARD) and explicitly documented that **the codebase does not post-process buyer text to force compliance** — nudge replies should be treated as LLM samples, not guaranteed policy execution. I also confirmed that award_contract runs only after human CLI approval regardless of buyer prose, so execution authority stays with the operator even when the buyer's output looks like an award.

6. Human-readable scenario log files (design decision that enabled the evaluation work)

I decided that each scenario run should produce a **human-readable log file** alongside the structured evidence_log_*.json — same content, formatted for a person to read top to bottom. This was a deliberate design choice, not a nice-to-have. The JSON files are the source of truth for programmatic evaluation, but they are not practical for reading turn-by-turn transcripts, spotting leakage in buyer prose, or tracing cause-and-effect across a run.

This decision had direct impact on Phase 3 outcomes. The leakage findings in finding 1, the Sonnet-snapshot observations in finding 2, the failure-mode analysis in finding 3, and the multi-year pricing trace in finding 4 all depended on being able to read transcripts fluently. Without human-readable logs, the hands-on evaluation work that produced this disclosure would not have been feasible at the depth I went.

7. Automation of evaluation artifacts

I was hands-on with the automation decisions for the evaluation pipeline — deciding which artifacts should be produced automatically per run (evidence logs, human-readable logs, per-scenario evaluation outputs) versus generated on demand, and how the pipeline should scale across the scenario set so that re-running the full evaluation did not require manual per-scenario bookkeeping. Cursor implemented the automation under my direction; the decisions about what to automate and how to structure the outputs are mine.

AI tools used in Phase 3

Claude Opus 4.7 — log analysis and final report synthesis

Phase 3 introduced a new pattern: I used **Claude Opus 4.7** as an analysis partner on the evidence logs and as a drafting partner for the final report.

How I used it: I fed Opus 4.7 the relevant log material and scenario context, and **asked specific, directed questions to shape the analysis** — what patterns appeared across runs, how to characterize a given emergent behavior, how to frame a finding for a specific audience section of the report. **I then reviewed and revised further, both the structure and the content.** The questions, the framing, and the final structure are mine; Opus 4.7 helped me surface and articulate.

What this means for verification: because I drove the questions and reviewed outputs against the logs, claims in the final report and in this document are grounded in the evidence logs, not in model recollection. Where Opus 4.7 produced a framing I kept, I verified the underlying claim against the specific log or code file referenced.

Cursor — coding agent

I continued to use **Cursor** as a coding agent in the IDE for Phase 3 implementation work: drafting and refactoring Python, navigating the repo, implementing the `_PUBLIC_BROADCAST_BUYER_APPEND` duplication, the `buyer_private_brief` plumbing through `orchestrator.py`, the `scenario_relax_public_broadcast` flag handling, and related changes. Cursor also continued as a **sparring partner** for implementation options and debugging.

What Cursor did not do in Phase 3: the evaluation analysis, the leakage diagnosis, the Sonnet-snapshot cause-and-effect reasoning, and the failure-mode analysis described above. Initial Cursor-drafted evaluation reports and test-case summaries were **disjointed and lacked the analytical structure and evaluation filters a human reviewer needs**. I restructured them, drove for strategic narrative and usability, and re-grounded them in the logs. Where Cursor's code touched evaluation or reporting logic, I verified the output against raw `evidence_log_*.json` files rather than trusting the generated summary.

Claude Opus 4.6 — ad-hoc questions

Used occasionally for ad-hoc questions, consistent with Phase 2.

Reconciliation with earlier phases

Phase 1 referenced ChatGPT and Gemini for ideation-stage brainstorming and execution-process education. **Those tools were not used in Phase 3.** Phase 3 AI usage was limited to Cursor (coding), Opus 4.7 (log analysis and report synthesis), and Opus 4.6 (ad-hoc).

Verification

The scenario runs, log analysis, and system testing described in findings 1–5 above are central to my verification work for Phase 3 — I inspected the dialogue while running the scenarios, which is how I confirmed what the system actually does and how I caught the leakage and snapshot issues. Beyond that, where it mattered, I spot-checked Opus 4.7's analytical framings against the underlying evidence logs before keeping them, and revised or dropped framings that did not hold up. The `model` field in each `evidence_log*.json` supports snapshot traceability for any specific run a reviewer wants to check.

The design choices, the scenario set, the governance framework, the evaluation analysis, and the causal reasoning that ties them together are my own. AI-assisted tooling implemented the code that instantiates them and helped articulate the findings after I made them.